Sonic AI take-home - technical explanation

1. Overview

I built an accounts-team app for fictional AcmeCorp: browse org/customers/invoices, chase overdue balances by email with an LLM-assisted collections agent, and suggest bank-statement -> invoice matches for human approval. Run with cp .env.example .env, fill mail/LLM credentials, then docker compose up --build. UI: http://127.0.0.1:3000. API: http://127.0.0.1:8000.

2. Architecture

```
Next.js (browser) --HTTP--> FastAPI routes --> services --> repositories --> MongoDB
                                                \-> adapters --> SMTP / IMAP / LLM
```

Layer	Role
Frontend	Next.js App Router pages; calls JSON APIs only
Routes	HTTP validation and response shaping
Services	Reminders, inbox pipeline, matching, seed, money/date helpers
Repositories	Mongo reads/writes
Adapters	SMTP send, IMAP fetch, OpenAI-compatible LLM client

Compose runs three services: mongo (named volume, healthcheck, not published), backend (Python 3.12, non-root, waits on mongo), frontend (Next standalone, waits on backend). Published ports bind to 127.0.0.1 only. The frontend never touches the database, mail server, LLM, or secrets.

3. Tech stack and why

Choice	Why
FastAPI	Typed routes, fast to structure as routes -> services -> repos
Next.js	Simple multi-page UI talking only to the API
MongoDB 7	Assignment preference for NoSQL; flexible docs for conversations/messages
rapidfuzz	Token-set + Jaro-Winkler for messy bank narrations without a model
OpenAI-compatible LLM	One adapter; swap Groq/OpenRouter/OpenAI via env
Docker Compose	Evaluator one-command run

4. Data model

Collections: organization, customers, invoices, conversations, messages, payments, payment_claims, bank_statement_uploads, bank_statement_rows, meta.

Money is stored as integer minor units (x100) via Decimal conversion at seed - no float arithmetic. Overdue is never stored; it is computed as status=sent, balance_due_minor>0, and due_date < as_of. DEMO_AS_OF_DATE overrides as-of when set; otherwise UTC today.

5. Collections agent

Reminders: one SMTP email per customer listing all overdue invoices; REMINDER_COOLDOWN_MINUTES (default 5); batch send returns sent / skipped reasons; per-customer force send; cooldowns cleared on demo reset.

Reply pipeline (when IMAP works): idempotent on Message-ID -> thread match -> match sender email to a customer in code before any LLM call -> unknown sender -> Needs attention with no LLM. Context is only that customer?s sent unpaid invoices plus org bank/policies. LLM returns JSON (intent, confidence, intents_detected, draft, optional payment_claim); parse strips fences, retries once, then Needs attention. Draft validator checks invoice numbers, amounts, and dates against DB (and org account digits). Routing: balance_query auto-send if confident+valid; payment_claim ack+note, never marks paid; extension_dispute_legal /

other_company_or_unknown / low confidence / multi-intent -> Needs attention. Humans can edit and send from Needs attention (re-validated). Prompt-injection: untrusted body delimiters, scoped context, JSON-only, validator gate.

IMAP limitation: outbound reminder SMTP is implemented. In my environment IMAP inbound fetch fails, so customer replies are not ingested automatically via Check inbox / the poller. The reply pipeline code is present but unusable until IMAP connectivity works for the evaluator.

6. Bank reconciliation

Local scorer only (no LLM for choosing invoices). Weights: invoice # in narration 40, payer name 30, amount 25, date 5. Suggest if score >= 55 (also >=38 with strong name + partial and clear gap). Ambiguity if top-second < 8 unless narration invoice # uniquely breaks the tie. Currency is a hard filter. TDS (INR + India only): credit = total - 10% of subtotal. Partial/overpay scored; CSV SHA-256 blocks duplicate uploads. Approval reloads the live invoice; underpay requires Close (TDS) or Leave open; at zero balance status becomes paid and reminders stop for that invoice. Same invoice cannot be the primary suggestion on two rows in one upload.

Why this design: truncated/reordered narrations fit token-set and Jaro-Winkler; wrong matches cost more than None, so precision beats recall; embeddings/LLM were skipped for core matching to keep results deterministic and reviewable.

Edge cases handled: Maple Retail -> None; R K Sharma -> None with INV-1017 amount hint; Saffron multi-invoice sum -> None with sum reason (no auto-split); same-amount / close scores (e.g. Kestrel) -> ambiguous None with candidates for one-click choose.

7. Security (implemented)

Pydantic/EmailStr on inputs; Mongo filters use typed values (no string-built queries); CSV header/size/row checks + SHA-256 dupe; CORS allowlist from env; LLM/mail context scoped per matched customer; secrets only in .env; ports on 127.0.0.1; Mongo unpublished; email bodies rendered as React text (escaped, no dangerouslySetInnerHTML). No authentication (localhost evaluation only).

8. How it was built

Spec-driven: product rules in specs/, then implementation with Cursor. Automated tests today: 3 pytest cases in backend/tests/test_matcher.py covering exact match, R K Sharma hint, and Saffron multi-sum reason. Manual Docker UI checks for Part A/C and outbound reminders.

9. Status and limitations

Area	Status
Part A (org, customers, invoices, reset)	Done
Reconciliation matcher + approve/reject/choose/TDS	Done
Reminder send, LLM classify/validate/route, Needs attention	Done
IMAP inbound fetch / auto-ingest replies	Partial - fails in my environment; not fixed
Rate limiting	Not done
Bonus: multi-invoice approve split	Not done (sum mentioned in reason only)
Bonus: payment-email ? bank row link	Not done
Bonus: OOO / bounce detection	Not done
Authentication	Not done (known limitation for local eval)